How-To Library

Informatica

Performance Tuning and Best Practices for File Ingestion and Replication Tasks

© Copyright Informatica LLC 2020, 2026. Informatica, Informatica Cloud, and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html.

Abstract

When you run file ingestion and replication tasks, you can tune multiple factors such as hardware, database, Amazon EC2, Secure Agent, and task parameters that impact the connector performance. This article describes general reference guidelines to help you tune these parameters before you configure file ingestion and replication tasks.

Supported Versions

- Data Ingestion and Replication

Table of Contents

Overview. . . . 2
Performance tuning areas. . . . 2
EC2 best practices. . . . 3
Tune the Secure Agent. . . . 3
JVM heap size. . . . 3
Tune the database. . . . 3
File size. . . . 3
Dedicate separate warehouses for Snowflake load and query operations. . . . 3
Databricks DB SQL cluster size. . . . 4
Tune the task. . . . 4
Source file size. . . . 4
Batch size. . . . 6

Overview

Performance tuning is an iterative process in which you analyze the performance, use guidelines to estimate and define parameters that impact the performance, and monitor and adjust the results as required.

This document describes the key hardware, database, Amazon EC2, Secure Agent, and file ingestion and replication task parameters that you can tune to optimize the connector performance when you run tasks.

This document also includes case studies that involve the tuning specifications required when you configure a file ingestion and replication task for sources and targets supported for Data Ingestion and Replication. The graphs constructed from the case studies illustrate the impact that the tuning has on the connector performance.

Note: The performance testing results listed in this article are based on observations in an internal Informatica environment using data from real-world scenarios. The performance of the connector might vary based on individual environments and other parameters even when you use the same data.

Performance tuning areas

You can tune the following parameters to optimize the performance of a connector when running file ingestion and replication tasks:

- Databricks cluster
- Amazon EC2

- Secure Agent
- Database
- File ingestion and replication task

EC2 best practices

To optimize the performance, consider the following best practices for Amazon EC2:

- Ensure that the Secure Agent hosted on the EC2 machine is in the same region as the database from where you want to read or write data.
- Choose the database in the region where the Amazon EC2 is located.
- Choose the right EC2 instance type based on your requirements.

For more information about the EC2 best practices, see the following website:
https://d0.awsstatic.com/whitepapers/aws-amazon-emr-best-practices.pdf

Tune the Secure Agent

You can tune the JVM heap size and the client memory limit for the Secure Agent to optimize the performance of a file ingestion and replication task.

JVM heap size

You can specify the amount of memory that the Secure Agent uses for the file ingestion and replication task. Default is 64 MB.

Tune the database

You can tune the database components to optimize the performance of a file ingestion and replication task.

File size

The number of load operations that run in parallel must not exceed the number of data files that you want to load. To optimize the number of parallel operations for a load, the size of the data must be between 500 MB and 1024 MB. The number of data files that are processed in parallel is determined by the batch size.

For example, if the source file size is 100 GB, specify the batch size as 20 where the size of each file is 500 MB.

Before you use Snowflake Cloud Data Warehouse V2 Connector to run a file ingestion and replication task to Snowflake, see the following website for information about how to prepare your files for loading data to Snowflake:
https://docs.snowflake.net/manuals/user-guide/data-load-considerations.html

Dedicate separate warehouses for Snowflake load and query operations

When you configure a Snowflake Data Cloud mapping to load large data sets to Snowflake, the query performance can get impacted. To optimize performance when you configure load and query operations, you must dedicate separate warehouses. The number and capacity of servers in a warehouse determine the number of data files that can be processed in parallel.

You must follow the file sizing guidelines so that the data load uses minimal resources. For more information, see topic "Preparing Your Data Files" in the following website:
https://docs.snowflake.net/manuals/user-guide/data-load-considerations-prepare.html

You can split larger data files to allow the load to scale linearly. A smaller warehouse is generally sufficient unless you want to concurrently load a large number of data ranging from a few hundreds to thousands of files. A larger warehouse such as X-Large or 2X-Large consumes more credits and might not optimize the performance.

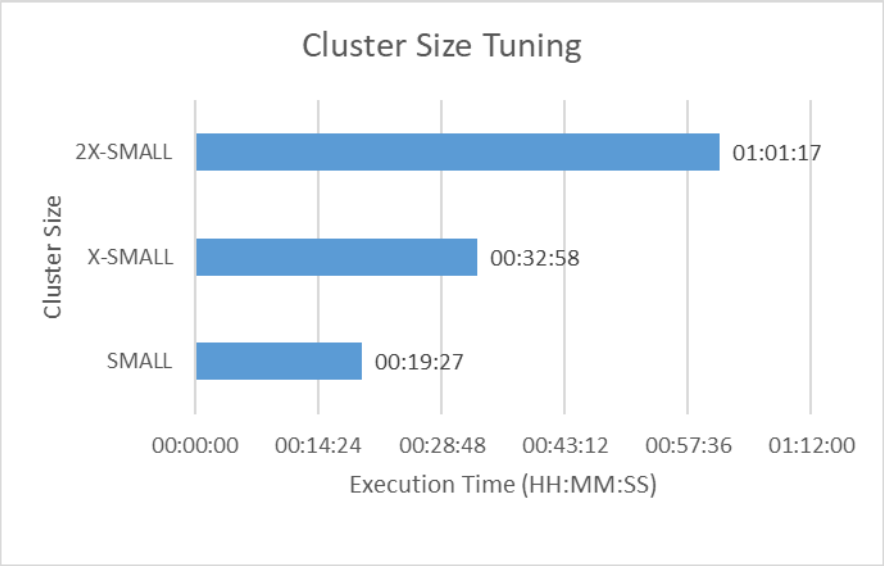
To understand the impact of dedicating warehouses to process queries, refer to the Snowflake guidelines and make appropriate changes depending upon the complexity of data and the amount of data that you want to process.

Databricks DB SQL cluster size

When you configure a Databricks DB SQL file in a file ingestion and replication task to load large data sets to Databricks, the query performance can get impacted. To optimize performance when you configure load and query operations, you must dedicate separate warehouses. The number and capacity of clusters in a warehouse determine the number of data files that can be processed in parallel.

You can split larger data files to allow the load to scale linearly. A smaller warehouse is generally sufficient unless you want to concurrently load a large number of data ranging from a few hundreds to thousands of files. A larger warehouse such as X-Large or 2X-Large consumes more credits and might not optimize the performance.

To understand the impact of database cluster size to process queries, refer to the guidelines and make appropriate changes depending upon the complexity of data and the amount of data that you want to process.



In the example, the source is Microsoft Azure Data Lake Storage Gen2 and the target is Databricks that resides on Microsoft Azure.

You can see that the performance of the file ingestion and replication task while writing to a Databricks target with different cluster size showed considerable performance improvement. The performance improved by nearly 3X.

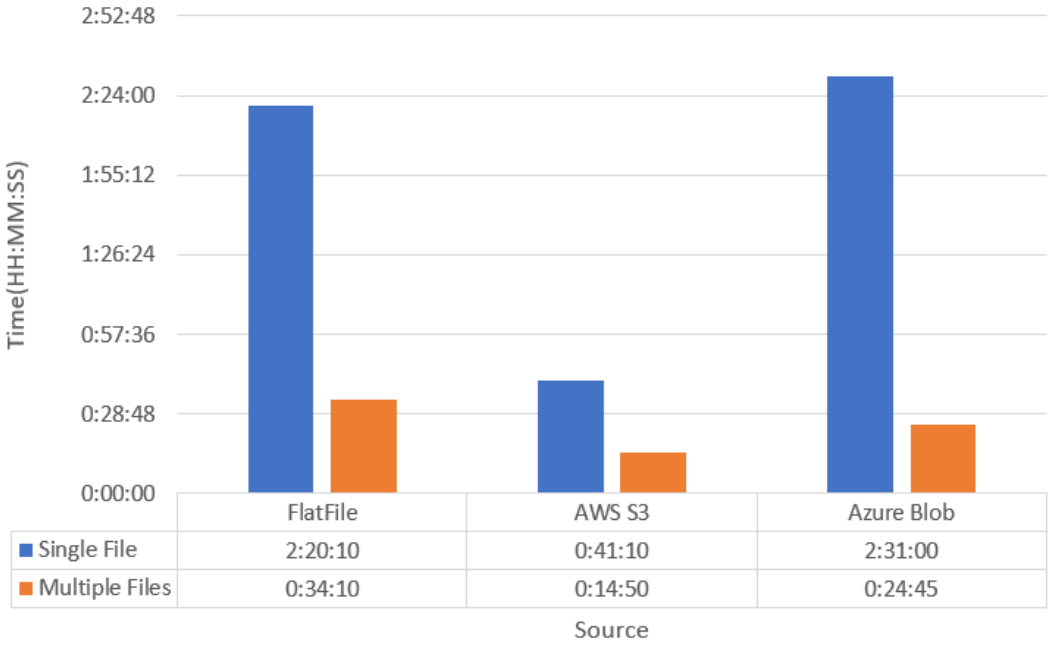
Tune the task

You can configure an appropriate file and batch size to optimize the performance of file ingestion and replication tasks.

Source file size

When the source files are large, consider splitting the sources file into multiple small files. When writing to a database, processing multiple small files is faster than processing a single large file. The recommended ideal size of a split file must be around 500 MB to 1 GB.

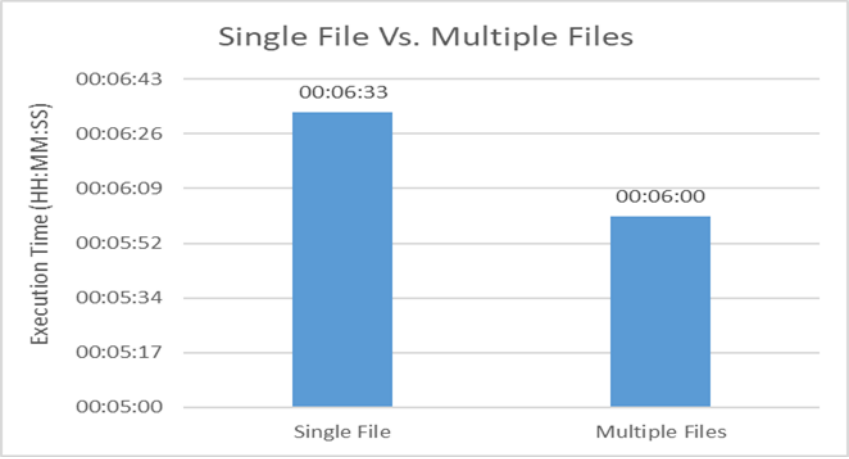
The following graph illustrates the impact of write performance from a single large source file in comparison with multiple small source files:



In the example, the target is Snowflake that resides on Amazon Web Services.

You can see that the performance of the file ingestion and replication task while writing to a Snowflake target from multiple smaller local flat file, Amazon S3, or Azure Blob sources showed considerable performance improvement. The writer performance improved by nearly 3X to 12X.

In the following example, the source is Microsoft Azure Data Lake Storage Gen2 and the target is Databricks that resides on Microsoft Azure.



You can see that the performance of the file ingestion and replication task while writing to Databricks target from multiple smaller files showed considerable performance improvement.

Batch size

When you configure a file ingestion and replication job to write to a target, the number of files that you want to write with each COPY command to the target impacts performance.

Specify a batch size for the maximum number of files to be transferred in a batch in the source properties of the file ingestion and replication task. When you specify a batch size, the performance of the task is optimized.

The default batch size is 5. When you use Snowflake Data Cloud Connector to write from Amazon S3 or Azure Blob Storage sources to a Snowflake target, you can specify a maximum batch size of 1000 in the Amazon S3 V2 or Azure Blob Storage V3 source properties.

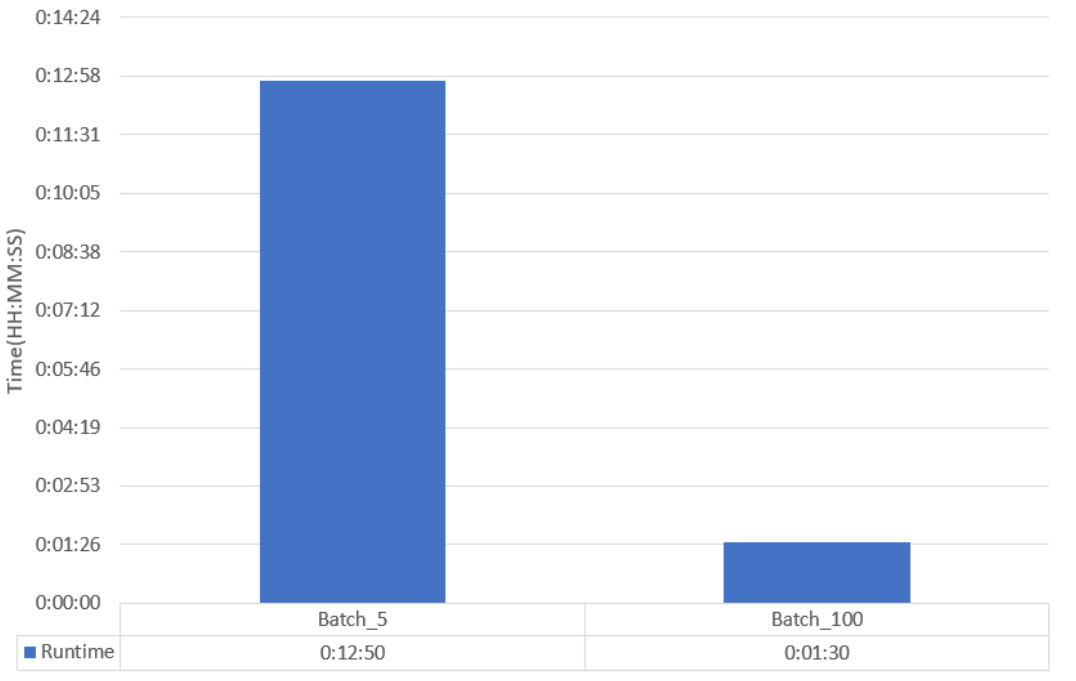
Similarly, when you use Databricks DB SQL Connector to write from Amazon S3 or Azure Blob Storage sources to a Databricks target, you can specify a maximum batch size of 1000 in the Amazon S3 V2 or Azure Blob Storage V3 source properties.

For other supported sources in file ingestion and replication tasks, you must specify a batch size between 1 and 20.

The following examples illustrate the optimization of a file ingestion and replication task when you configure a batch size to write to a target.

Impact of batch size on job performance - Amazon S3 source and Snowflake target

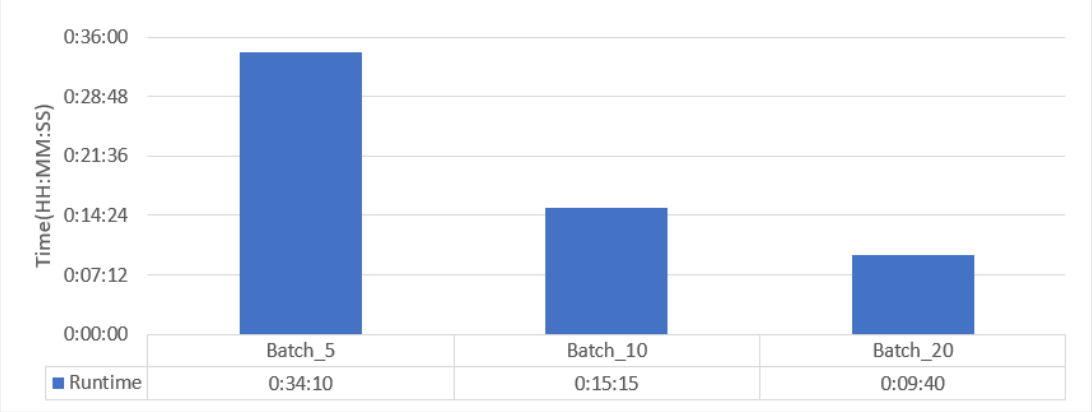
The following graph depicts the impact of the batch size on the performance of a file ingestion and replication task when the source is Amazon S3 and the target is Snowflake:



You can observe an improvement of around 10X in the writer performance when you specify an appropriate batch size.

Impact of batch size on job performance - Local Flat File source and Snowflake target

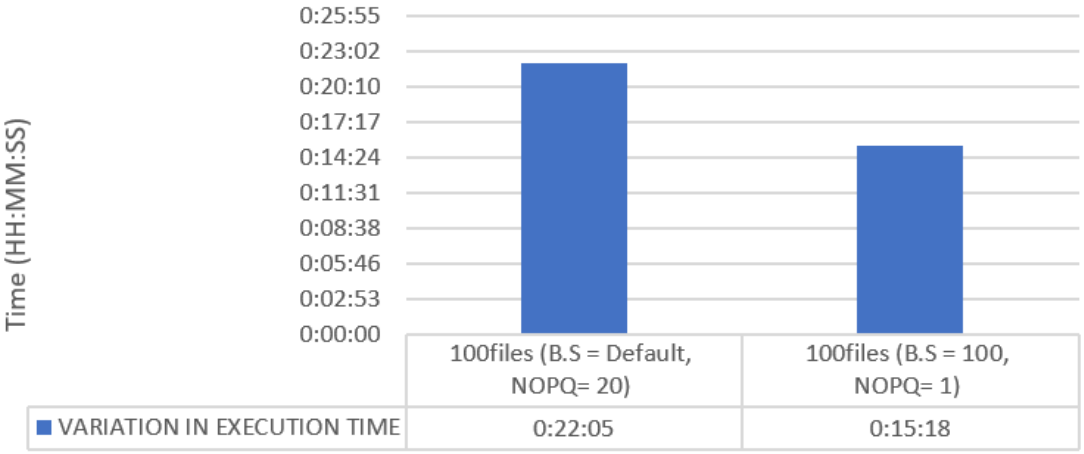
The following graph depicts the impact of the batch size on the performance of a file ingestion and replication task when the source is a local flat file and the target is Snowflake:



You can observe around 4X improvement in the performance with an increase in the batch size.

Impact of batch size on job performance - Azure Blob source and SQL Data Warehouse target

The following graph illustrates the impact of the batch size on the performance of a file ingestion and replication task when the source is Azure Blob and the target is SQL Data Warehouse:

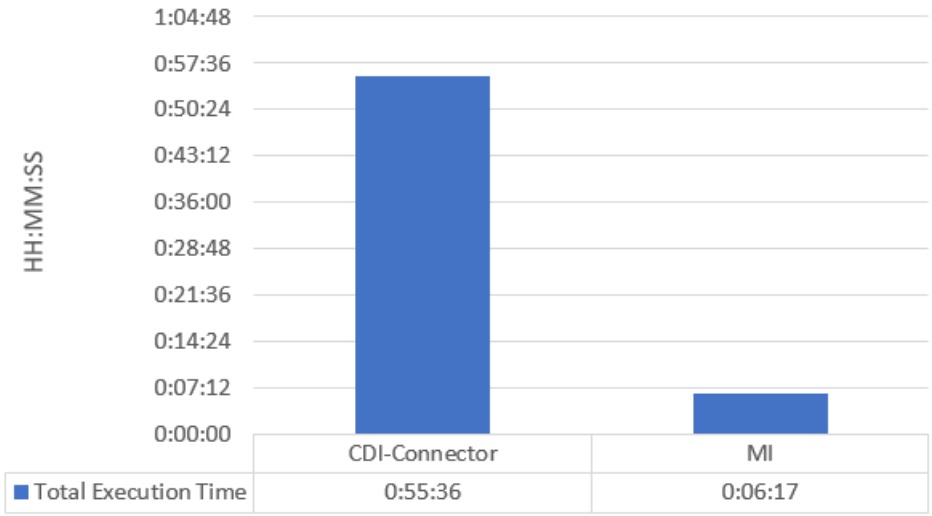


NOPQ (Number of Polybase Queries) = Number of Files/Batch Size.

You can observe around 2X improvement in the writer performance.

Performance impact between a file ingestion and replication task and a mapping task - Flat File source and Microsoft Azure Data Lake Store target

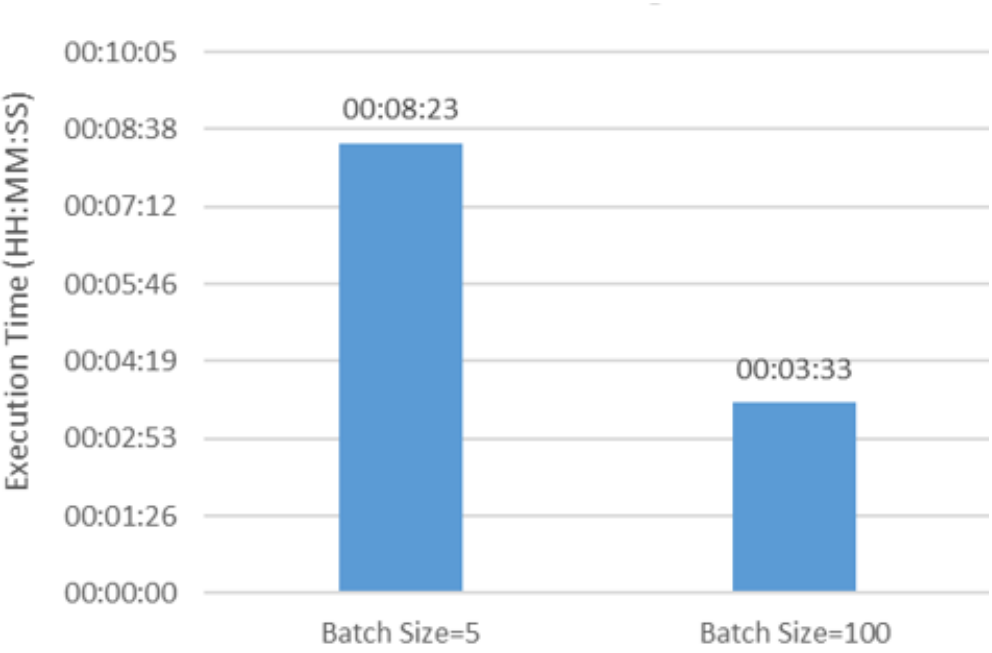
The following graph illustrates the impact of the batch size in a mapping task in Data Integration and a file ingestion and replication task in Data Ingestion and Replication when the source is Azure Blob and the target is SQL Data Warehouse:



The performance of the file ingestion and replication task is 9X times faster than a mapping task.

Impact of batch size on job performance - Microsoft Azure Data Lake Storage Gen2 source and Databricks target

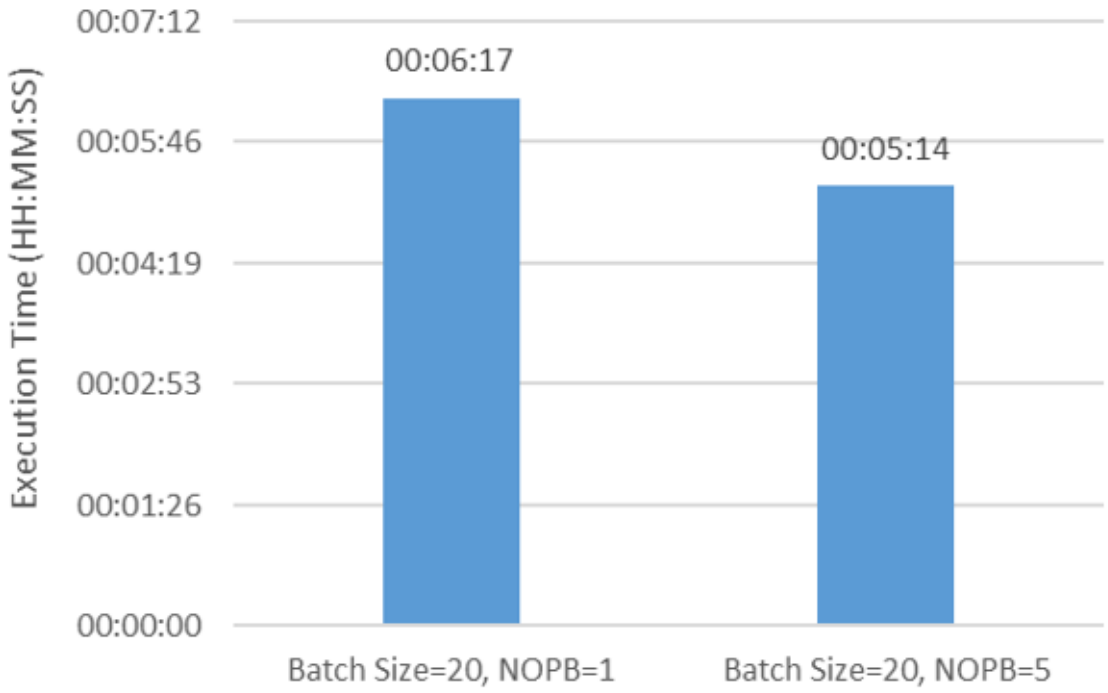
The following graph depicts the impact of the batch size on the performance of a file ingestion and replication task when the source is Microsoft Azure Data Lake Storage Gen2 and the target is Databricks:



You can observe an improvement of around 2X in the writer performance when you specify an appropriate batch size.

Impact of tuning number of parallel batches along with batch size

The following graph illustrates the impact of tuning number of parallel batches with batch size on the performance of a file ingestion and replication task when the source is Microsoft Azure Data Lake Storage Gen2 and the target is Databricks DB SQL:



Where NOPB is the number of parallel batches.

You can observe a significant improvement in the writer performance.

Guidelines for optimizing the file ingestion and replication task performance to a Microsoft Azure Data Lake Store target

Consider the following guidelines to optimize the performance of the file ingestion and replication task while uploading files to a Microsoft Azure Data Lake Store target:

- The batch size must not exceed a maximum size of 20 when the source is a flat file.
- The batch size must be less than or equal to the number of VCores in the agent virtual machine.
- To optimize performance, the recommended JVM maximum heap size must be 1 GB to avoid garbage collection overhead and out-of-memory errors. When you use the default JVM heap size, the task might fail with an out-of-memory error. If you do not specify an appropriate JVM maximum heap size, the performance of the task might also be impacted.

Guidelines for optimizing the job performance to a Databricks DB SQL target

Consider the following guidelines to optimize the performance of the file ingestion and replication task while writing data to a Databricks DB SQL target:

- The batch size must not exceed a maximum size of 1000 when the source is a Microsoft Azure Data Lake Storage Gen2/S3.
- For large datasets, bigger size clusters gives better performance.

General guidelines for configuring the batch size

Consider the following guidelines to optimize performance:

- When you specify a batch size, you must also ensure that the source file is split into multiple smaller files so that performance is optimized.
- For optimized performance, batch size should be equal to or close to the number of files to be loaded in the target.
- Increasing the number of parallel batches with batch size can further improve performance.
- For information about the appropriate batch size value that you can specify for different sources and targets in a file ingestion and replication task, see the Local folder source properties topic in the *File Ingestion and Replication* documentation.

Authors

Siddiq Hussain

Dimple Rai